AOSP 进阶丛书 · 第一册 · ANDROID 13 (API 33)

# Android 13 全栈架构总览

基于 AOSP Android 13 (Tiramisu) 完整源码的全景导读：五层架构、源码树地图、HAL 三代演进、构建与分区体系、ART 运行时、SELinux 与属性系统——每个论断附真实源码路径与行号。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

# 目 录


前言：从应用开发者到系统开发者 . . . . . 3

第一章 五层架构总论 . . . . . 4

第二章 源码树全景导览 . . . . . 6

第三章 内核层与 HAL 层 . . . . . 7

第四章 Native 世界：守护进程与基础库 . . . . . 10

第五章 Framework 层：API、服务与分发中枢 . . . . . 12

第六章 层间通信机制总览 . . . . . 15

第七章 构建与产物体系 . . . . . 16

第八章 运行时体系：ART 与 Zygote . . . . . 19

第十章 深入属性系统：一次 setprop 的完整旅程 . . . . . 22

第十一章 SELinux：系统的第二套权限体系 . . . . . 25

第十二章 图形与输入骨架概览 . . . . . 26

第十三章 观测与调试：把系统打开给你看 . . . . . 26

第十四章 进阶路线图：本丛书之后怎么走 . . . . . 28

第十五章 设计决策分析（全书总结） . . . . . 28

第十六章 深水区专题一：SELinux 策略实战与 APEX 内部 . . . . . 29

第十七章 深水区专题二：Soong 实战与链接器命名空间 . . . . . 30

第十八章 边境机制三讲：JNI、Context 家族与 native 智能指针 . . . . . 31

附录 A 版本指纹与阅读路线 . . . . . 32

附录 B 系统进程归属速查 . . . . . 33

附录 C 常用系统属性速查（带源码出处） . . . . . 33

附录 D 术语表 . . . . . 34

附录 E 上手实验（半天量） . . . . . 35

教科书习题集（分级） . . . . . 35

术语表（按拼音排序） . . . . . 37

排障手册（按"我的问题是..."索引） . . . . . 38

**本册速览**

五层架构与贯穿机制（Binder/属性/SELinux/APEX）——任何模块 30 秒定位
源码树地图 · 分区与构建（Soong/APEX/AVB） · HAL 三代演进 · ART 三级执行
深水区：SELinux 策略实战 · apexd 回滚 · Soong 写模块 · 链接器命名空间
边境三讲：JNI 三类引用 · Context 家族 · sp/wp/RefBase · Handler/Looper
建议顺序：通读建框架 → 对照锚点精读 → 附录 E 实验 → 进第二册

# 前言：从应用开发者到系统开发者

本书是《AOSP 进阶丛书》的第一册，基于你本机的 **AOSP Android 13（Tiramisu，API 33，安全补丁 2022-08-05）** 完整源码树撰写，所有结论都给出了 **文件:行号** 级别的源码锚点，可在 `/run/media/marcusgu/SoftWare/aosp` 下逐一验证。

作为应用开发者，你已经熟悉 SDK 的世界：Activity、Service、Handler、Gradle。而系统开发者的世界从这里开始向下延伸——你的代码之下的 `framework.jar` 是怎么运行起来的，`getSystemService` 拿到的到底是什么，开机 logo 之后到 Launcher 出现之前发生了什么。本丛书沿着“**开机 → 系统就绪 → 应用启动 → 进程间通信**”这条贯穿全栈的主线组织内容：

- **第一册（本书）**：全景地图。五层架构、源码树导览、构建与分区体系、运行时体系。读完你将获得“任何模块都能定位到它属于哪一层、住在哪个目录、依赖谁”的能力。
- **第二册**：系统启动主线。init → Zygote → SystemServer 的逐函数精读。
- **第三册**：Binder IPC 全链路。从内核驱动到 Java 层的完整调用链剖析。

阅读方法建议：先通读建立框架，再对照源码锚点精读；每册末尾的“设计决策”一章是面试与架构能力的分水岭，值得反复咀嚼。

## 先修要求与学习路线

**先修**：熟悉 Java/Kotlin Android 应用开发（Activity 生命周期、Service、Handler 基本用法）。不需要 C++ 或 Linux 内核基础。

**读完本书你将能够**：

- 在 30 秒内定位任何 AOSP 模块属于哪一层、住在哪个目录、依赖谁
- 画出 Android 五层架构图并解释每层的职责与边界
- 理解 Binder/属性/SELinux/APEX 四条贯穿机制各自解决什么问题
- 读懂 Android.bp 构建文件并理解 Soong 构建管线
- 独立完成附录 E 的五个动手实验
- 为后续第二册（启动主线）和第三册（Binder）建立完整的知识地图

**本书不覆盖**（后续册深入）：进程调度细节（书4）、窗口管理（书5）、图形渲染管线（书8）、Binder 协议逐跳分析（书3）。

> **设计洞察 · 本丛书的源码约定**
>
> 本书所有行号基于 Android 13（TP1A）源码树实测；目录路径均相对源码树根 `$A`。源码树严格只读——研读系统代码的首要纪律是：先理解，再动手。

# 第一章 五层架构总论

**设计洞察 · 学习目标**

1. 画出五层架构图；2. 解释每层核心职责与依赖方向；3. 说出三条贯穿机制；4. 追踪一行代码从App到内核

Android 的软件栈通常被划分为五层。分层的意义不在于“好看”，而在于**依赖方向的确定性**：每一层只依赖它下面的一层，越往上越贴近应用，越往下越贴近硬件。

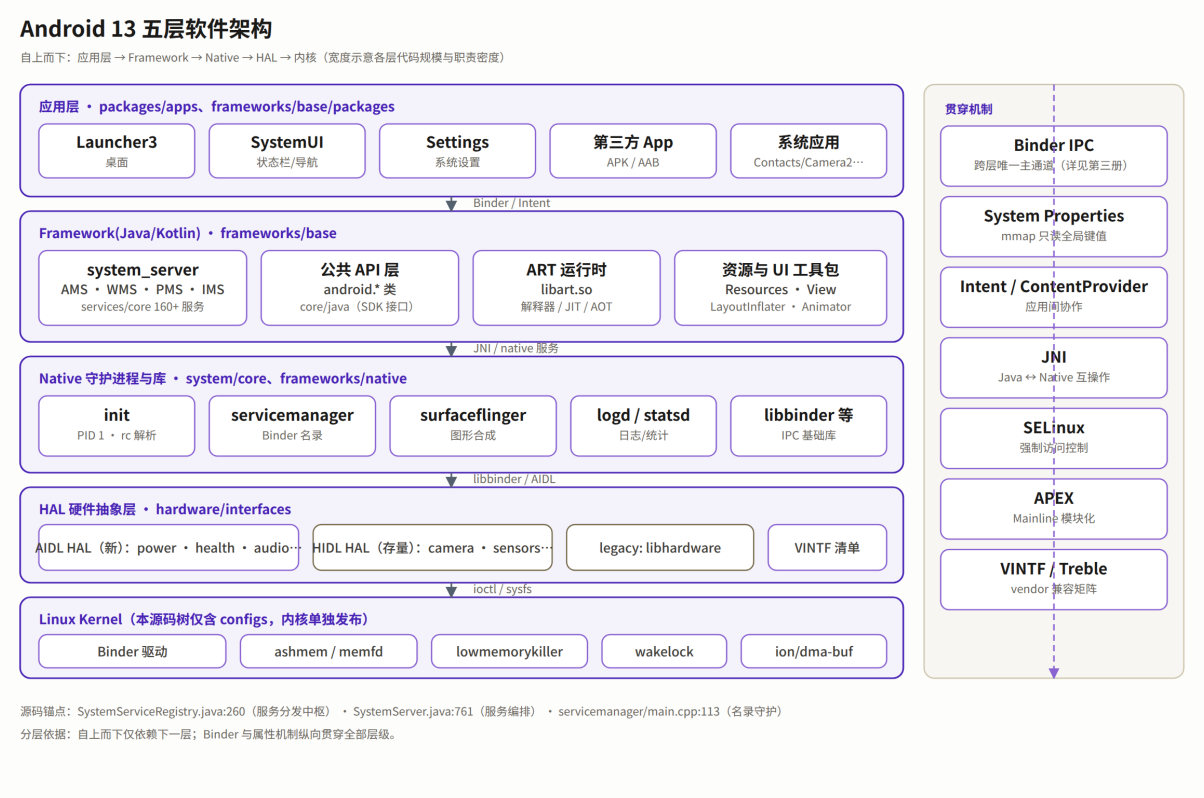



图 1 Android 13 五层软件架构总图：左侧自上而下为应用层、Framework、Native、HAL、内核，右侧为纵向贯穿的公共机制

## 1.1 自上而下看五层

**应用层**运行在独立的 App 进程中，通过 SDK API 与系统交互。系统应用大多住在源码树两处：`packages/apps/`（Launcher3、Settings、Camera2 等）与 `frameworks/base/packages/SystemUI`。它们与第三方 App 本质相同——都靠 Binder 调用系统服务，只是签名与权限不同。

**Framework 层**是系统开发者的主战场，物理上由 `frameworks/base` 承载。它包含两部分：公共 API（`core/java/android/`，即编译进 `framework.jar` 的 `android.*` 类）与系统服务实现（`services/core/java/com/android/server/`，编译进 `services.jar`，运行在 system_server 进程）。你调用的每个系统 API，最终几乎都穿过前者、经 Binder 落到后者。

**Native 层**是 C/C++ 的世界：`system/core` 与 `frameworks/native` 中的守护进程（init、servicemanager、surfaceflinger、logd 等）和基础库（libbinder、libutils、libcutils）。渲染合成、输入分发这类对性能极敏感的职责都在这一层完成。

**HAL（硬件抽象层）**把“芯片相关”与“系统无关”隔离开：接口定义集中在 `hardware/interfaces/`，实现由各 SoC 厂商提供在 vendor 分区。Android 13 的主线是 AIDL HAL，存量 HIDL 仍大量并存。

**内核层**是 Linux kernel 加上 Android 特有补丁（Binder 驱动、ashmem/memfd、lowmemorykiller、wakelock 等）。值得注意的是：**本源码树的 `kernel/` 目录只有 configs、prebuilts、tests 三个子目录，不含内核源码**——AOSP 的内核是独立仓库（如 android13-5.10），但内核的稳定 UAPI 头（包括 Binder 协议的 `binder.h`）通过 `external/kernel-headers/original/uapi/` 进入编译。

## 1.2 右侧“贯穿列”：真正的系统观

初学者容易把五层理解成“五块积木”，但真正的架构能力在于看见**纵向贯穿的机制**：

- **Binder IPC** 几乎是所有跨层调用的唯一通道：App → system_server 是 Binder，system_server → HAL 服务还是 Binder。第三册将逐跳剖析它。
- **System Properties** 是全机共享的键值区：init 建立，所有进程 mmap 只读共享，`persist.` 前缀落盘。它是比 Binder 轻得多的“单向广播”。
- **SELinux** 的策略覆盖从 init 第一个动作到每个服务的每一次访问——`system/sepolicy/` 是策略源码，理解它才能理解“为什么这个服务连不上那个端口”。

## 1.3 实战：一行代码看穿五层

以应用里最常见的调用为例——`powerManager.isInteractive()`（判断屏幕是否点亮）：

java 应用进程内（五层穿越起点）

```
PowerManager pm = context.getSystemService(PowerManager.class);
boolean lit = pm.isInteractive();
```

它的下行路径是：`PowerManager`（应用层 API，`core/java/android/os/PowerManager.java`）持有一个 `IPowerManager` Binder 代理 → `SystemServiceRegistry` 在首次获取时经 `ServiceManager.getService("power")` 从 **servicemanager**（Native 层）查到 system_server 注册的电源服务 Binder 句柄 → 调用发起后经 **Binder 驱动**（内核层）进入 system_server（Framework 服务层）的 `PowerManagerService.isInteractive()`（`services/core/.../power/PowerManagerService.java`）→ 返回值原路回到应用。一行代码，五层全穿。**本丛书剩余两册，就是把这条线上的每一层拆开讲透。**

**设计洞察 · 分层的本质是依赖管理**

五层中任何一层被替换，只要接口契约不变，上下层不受影响。这正是 Treble 工程的落点：把 vendor 相关的 HAL/内核钉死接口，让 system 分区可以独立升级。判断一个改动“该放在哪一层”的系统思维：能否被多个应用复用 → Framework；是否性能关键且稳定 → Native；是否因硬件而异 → HAL。

# 第二章 源码树全景导览

**设计洞察 · 学习目标**

1. 说出五个核心目录；2. 30秒定位任意函数；3. 判断新模块该放哪层

面对数 GB 的源码树，系统开发者的第一项能力是“看目录知职责”。本章给出一张地图，并标出本丛书的精读路径。




图 2 AOSP 源码树地图：上半部分为核心目录分组，下半部分为“从开机到 App”主线的落点

## 2.1 必须烂熟于心的五个目录

| 目录 | 职责 | 关键子目录 |
| --- | --- | --- |
| `frameworks/base` | Framework 全部 | core/java（API）、services（服务端）、cmds/app_process（Zygote） |
| `system/core` | 基础组件 | init、rootdir（init.rc）、debuggerd、libutils |

| 目录 | 职责 | 关键子目录 |
| --- | --- | --- |
| frameworks/native | 图形与 IPC | libs/binder（libbinder）、cmds/servicemanager、services/surfaceflinger |
| hardware/interfaces | HAL 接口定义 | power、audio、health 等 AIDL/HIDL 接口与 VINTF 矩阵 |
| art | 运行时 | runtime（虚拟机）、dex2oat（AOT 编译器）、compiler |

其余目录按需检索：packages/（系统应用）、bionic（Android 版 libc）、libcore（Java 核心库，含 OpenJDK 移植 ojluni）、build/（构建系统）、external/（数百个第三方项目）、prebuilts/（预置工具链）。

## 2.2 用目录回答三个问题

**“servicemanager 在哪？”** 它在 frameworks/native/cmds/servicemanager/（本树位置；较老教材写 system/core/servicemanager，Android 13 已迁移）。这提醒我们：**教材会过时，目录检索能力不会**。

**“init.rc 有几份？”** 入口是 system/core/rootdir/init.rc，它 import 了 init.${ro.hardware}.rc、init.${ro.zygote}.rc 等；每个守护进程还各自带 /system/etc/init/*.rc。第二册将精读这条 import 链。

**“我该改哪层的代码？”** 先看现象的层级：API 行为问题查 frameworks/base/core/java 与 services/core；性能/图形查 frameworks/native；设备差异查 hardware/interfaces 与 device/。

> **源码锚点 · 快速检索工具链**
> <font face="MonoFT" size="9">ripgrep</font> 全树检索：rg -t cpp "SecondStageMain" --line-number<br/>按文件名定位：find . -name "ZygoteInit.java" -not -path "*/out/*"<br/>确认版本：build/make/core/version_defaults.mk（SDK_INT 33）<br/>排除产物：始终加 -not -path "*/out/*" 避免命中构建产物

# 第三章 内核层与 HAL 层

> **设计洞察 · 学习目标**
> 1. 列举五项内核增强；2. 解释HAL三代演进；3. 读懂HAL目录结构

## 3.1 Android 对 Linux 内核的增强

Android 内核 = 标准 Linux + 一组"Android 通用内核"补丁。最重要的五项：

- **Binder 驻动**（`drivers/android/binder.c`，不在本树）：进程间通信的基石，一次拷贝与内核级身份校验都由它实现（第三册详述）。UAPI 定义在 `external/kernel-headers/original/uapi/linux/android/binder.h`。
- **ashmem/memfd**：匿名共享内存，Binder 传递大块数据（如位图）的配套机制。
- **lowmemorykiller 演进**：内核态 LMK 已被用户态 `lmkd`（`system/core/lmkd`）+ PSI 压力信息取代，通过 socket 与 init 体系协同。
- **wakelock/wakeup source**：电源管理的同步原语，`libsuspend`（`system/core/libsuspend`）封装。
- **ION/DMA-BUF**：显存缓冲分配，现已被内核标准 DMA-BUF 取代。

## 3.2 HAL：三代演进

HAL 的历史是 Android 对“碎片化”宣战的历史。legacy HAL 把厂商代码编进 Framework 进程，升级系统必须重编厂商代码；Android 8 的 Treble 用 HIDL 强制进程隔离与版本化；Android 10 起 AIDL HAL 统一语言栈，让 HAL 与系统服务共用同一套 Binder 设施与工具链。




图 3 HAL 三代演进时间轴与 AIDL HAL 调用路径、VINTF 兼容矩阵

## 3.4 完整走读：以 Power HAL 为例看穿一个 AIDL HAL 的全部文件

以 `hardware/interfaces/power/aidl/` 为例——这是本树中最简洁的 AIDL HAL 参考实现：

### 文件一：接口定义（.aidl）

java hardware/interfaces/power/aidl/android/hardware/power/IPower.aidl

```
// 讲解注释①：接口文件是 HAL 的"合同"——
// framework 和 vendor 都依据它生成绑定代码
// :24
interface IPower {
// 讲解注释②：每个方法就是一条"合同条款"
void setMode(in Mode type, in boolean enabled);
```

```
bool isModeSupported(in Mode type);
int getHintSessionPreferredRate();
}
// 辅助类型也在同目录：
// Mode.aidl（枚举：INTERACTIVE/DOUBLE_TAP_TO_WAKE...）
// Boost.aidl（枚举：DISPLAY_UPDATE_IMMINENT...）
// WorkDuration.aidl（工作负载时长报告）
```

## 文件二：参考实现（default/Power.cpp）

cpp hardware/interfaces/power/aidl/default/Power.cpp

```
// 讲解注释③：厂商实现继承生成的 Binder 接口
// ndk::ScopedAStatus 是 AIDL C++ 的返回值包装
using ::aidl::android::hardware::power::IPower;
class Power : public BnPower {
ndk::ScopedAStatus setMode(Mode type, bool enabled) override {
// 讲解注释④：真正写硬件的代码在这里
// 例如：echo 1 > /sys/power/... 或 ioctl(...)
return ndk::ScopedAStatus::ok();
}
};
```

## 文件三：构建描述（default/Android.bp）

text default/Android.bp（五要素模板）

```
aidl_interface {
name: "android.hardware.power", // 模块名
vendor_available: true, // vendor 分区可用
srcs: ["android/hardware/power/*.aidl"],
}
cc_binary {
name: "android.hardware.power-service.example",
relative_install_path: "hw", // /vendor/bin/hw/
init_rc: [":android.hardware.power.rc"], // init 启动脚本
vintf_fragments: [":android.hardware.power.xml"], // VINTF 清单
vendor: true, // 归入 vendor 分区
srcs: ["Power.cpp", "main.cpp"],
shared_libs: ["libbinder_ndk", "liblog"],
}
```

## 文件四：init 脚本（.rc）

text .rc 文件（init 从这里启动 HAL 服务）

```
service vendor.power-hal /vendor/bin/hw/android.hardware.power-service.example
class hal // init 的 hal 类启动（先于 framework 服务）
user system
group system
seclabel u:r:hal_power:s0 // SELinux 域标签
```

## 文件五：VINTF 清单（.xml）

xml VINTF 清单（告诉 framework "这个设备支持什么 HAL"）

```
<hal format="aidl">
```

```
<name>android.hardware.power</name>
<fqname>IPower/default</fqname> // 实例名
</hal>
// framework 的 compatibility_matrices 里声明需要什么——
// 构建期对齐两份清单，不匹配 = 拒绝构建
```

### 文件六：SELinux 三件套

text SELinux（vendor/sepolicy/）

```
hal_power.te: // 域定义（allow 规则）
type hal_power, domain;
allow hal_power power_service:service_manager find;
file_contexts: // 文件标签
/vendor/bin/hw/android.hardware.power-service.example
u:object_r:hal_power_exec:s0
service_contexts: // 服务标签
android.hardware.power.IPower/default
u:object_r:hal_power_service:s0
```

**设计洞察 · 逐段精读：为什么要六个文件**

这是 Treble 的"合同精神"：接口（.aidl）是合同条款、实现（.cpp）是履约、构建（.bp）是打包、启动（.rc）是交付、VINTF 是公证、SELinux 是守卫。**少一个文件，这个 HAL 就无法被 framework 正确发现和使用**。这就是为什么书14 说"五步走一个 HAL"——每步对应这里的文件。

**设计洞察 · 为什么 HIDL 被 AIDL 取代**

HIDL 达成了隔离目标，但代价是第二套 IDL 编译器、第二个 binder 域（/dev/hwbinder）、第二个服务管理器（hwservicemanager）。AIDL HAL 复用主线设施后，工具链、调试手段、线程模型全部归一——工程上“少一套系统”的收益压倒了迁移成本。存量 HIDL（如部分 camera 接口）长期冻结维护，读代码时先看 `aidl/` 还是 `1.x/` 目录区分代际。

# 第四章 Native 世界：守护进程与基础库

Native 层的守护进程几乎全部由 init 在启动各阶段拉起（谁在什么阶段启动，第二册的触发链图给出完整答案）。本册先建立名录。

## 4.1 核心守护进程名录

| 进程 | 源码位置 | 职责 | 启动类别 |
| --- | --- | --- | --- |
| `init` | system/core/init | PID 1，解析 rc、孵化与监护所有服务 | 内核第一个用户态进程 |
| `servicemanager` | frameworks/native/cmds/servicemanager | Binder 服务名录（句柄 0） | class core |

| 进程 | 源码位置 | 职责 | 启动类别 |
| --- | --- | --- | --- |
| surfaceflinger | frameworks/native/services/surfaceflinger | 图形图层合成、VSync 分发 | class core |
| logd | system/logging/logd | 全机日志汇聚 | class core |
| lmkd | system/memory/lmkd | 低内存查杀 | class core |
| vold | system/vold | 卷管理、加密、fuse | early-fs 阶段 |
| netd | system/netd | 网络策略与 DNS | class main |
| audioserver / cameraserver / mediaserver | frameworks/av 等 | 媒体相关服务进程 | class main |
| bootanimation | frameworks/base/cmds/bootanimation | 开机动画（开机完成的视觉信号） | class core animation |

“类别”（class core/main/hal…）不是注释，而是 init 批量启动的调度单位——`class_start main` 一条命令拉起整组服务，详见第二册 `builtins.cpp` 的 `do_class_start`。

## 4.2 基础库版图

- **libbinder**（`frameworks/native/libs/binder`）：Native 侧 Binder 运行库，ProcessState/IPCThreadState 所在地，第三册主角。
- **libutils**（`system/core/libutils`）：RefBase、sp/wp 智能指针、Looper、Thread——Native 世界的“公共底座”。
- **libcutils**（`system/core/libcutils`）：属性读写、socket 工具、trace 等 C 工具集。
- **libgui / libui**（`frameworks/native/libs/{gui,ui}`）：BufferQueue、Surface、图形缓冲——渲染栈的 Native 骨架。

> **源码锚点 · 从守护进程到“服务类”的对应**
> <font face="MonoFT" size="9">servicemanager.rc</font>：class core animation，critical —— 名录自身也是被 init 管理的服务<br/><font face="MonoFT" size="9">init.zygote64.rc</font>：class main —— Framework 全部进程的祖先进“主类”<br/><font face="MonoFT" size="9">SystemServer.java:435</font>：sMaxBinderThreads=31 —— system_server 的 Binder 线程预算

## 4.3 启动性能观测的一件老兵器：bootchart

`system/core/init/bootchart.cpp` 是 init 内建的启动时间线采集器：设置 `persist.init.bootchart` 后，init 记录各 action/服务时间戳，配合 `system/core/init/grabbootchart.sh` 产出甘特图。它是第二册“触发链”一章的天然实验伙伴——改一个 rc 触发顺序，bootchart 立刻告诉你启动时间花在哪。Google 现代工作流更多用 perfetto（`external/perfetto`），但 bootchart 与 init 代码同源同寿命，读懂它等于复习 init 的数据结构。

# 第五章 Framework 层：API、服务与分发中枢

## 5.1 一份代码，两个 jar

Framework 的物理形态是两个部署单元：`framework.jar`（所有进程共享的 API 与基础类）与 `services.jar`（只在 system_server 进程加载的服务实现）。App 进程通过 Zygote 的写时复制共享 `framework.jar` 的内存映射——这是全机内存优化的大头（第二册 Zygote 章详述）。

**API 与实现的边界**藏在 `frameworks/base` 的目录结构里：`core/java/android/app/ActivityManager.java` 是给应用调用的门面，真正逻辑在 `services/core/java/com/android/server/am/ActivityManagerService.java`（1.8 万行）。门面通过 Binder 把请求送进 system_server。

## 5.2 getSystemService 的分发中枢

应用里再平常不过的一行 `context.getSystemService(Context.POWER_SERVICE)`，背后是 `frameworks/base/core/java/android/app/SystemServiceRegistry.java` 这个中枢：

> **源码锚点 · SystemServiceRegistry：服务注册表**
> <font face="MonoFT" size="9">SystemServiceRegistry.java:260</font> 类定义 —— 静态注册 100+ 服务的 fetcher<br/><font face="MonoFT" size="9">SystemServiceRegistry.java:270</font> SYSTEM_SERVICE_FETCHERS 全局 map<br/><font face="MonoFT" size="9">SystemServiceRegistry.java:284</font> CachedServiceFetcher：进程级缓存的管理器对象<br/><font face="MonoFT" size="9">SystemServiceRegistry.java:289+</font> 各服务以 registerServiceFetcher 注册获取逻辑

多数 fetcher 的逻辑是：拿 Context → 通过 `ServiceManager.getService("power")` 拿到 Binder 代理 → `IPowerManager.Stub.asInterface` 包装 → 缓存。也就是说，**管理器对象只是 Binder 代理的本地缓存马甲**。看懂这一点，你就打通了“应用层 API → Binder → system_server”的静态地图；动态的每一步在第三册。

## 5.3 system_server 里的服务版图

system_server 以单进程容纳 160+ 系统服务（第二册给出启动时序），按包名归类：`am/`（AMS 及进程/广播/服务管理）、`wm/`（WMS 与窗口/ATM）、`pm/`（PMS 包管理）、`power/`、`input/`、`display/`、`notification/`、`usb/`……而图形合成（SurfaceFlinger）、音频（audioserver）等因性能与崩溃隔离**有意留在外面**——集中与隔离的权衡，是第十章设计决策的主题之一。

## 5.4 常见系统服务速查表

| getSystemService 返回 | Binder 名 | 实现类（services/core/...） | 所在进程 |
|---|---|---|---|
| ActivityManager | activity | am/ActivityManagerService | system_server |
| WindowManager | window | wm/WindowManagerService | system_server |
| PackageManager | package | pm/PackageManagerService | system_server |
| PowerManager | power | power/PowerManagerService | system_server |
| InputManager | input | input/InputManagerService | system_server |
| DisplayManager | display | display/DisplayManagerService | system_server |
| NotificationManager | notification | notification/NotificationManagerService | system_server |
| LocationManager | location | location/LocationManagerService | system_server |
| AudioManager | audio | （跨进程）audio 服务 | audioserver |
| CameraManager | camera | （跨进程）camera 服务 | cameraserver |
| SensorManager | sensors | （跨进程）sensorservice | sensorserver |
| WifiManager | wifiservice | wifi/WifiService | system_server |

注意“所在进程”一列：多数框架服务集中在 system_server，但音频/相机/传感器有意独立进程——再次呼应“故障域”划分。第三册会以 `activity` 服务为例，把“Binder 名 → 代理对象 → 具体方法”的运行期解析全程展开。

## 5.5 system_server 服务全表（按启动批次分组）

这是全书信息密度最高的一张表——**每行一个服务，一眼看到 system_server 的全貌**。

### Bootstrap 批（依赖地基，:1052-1330）

| 序 | 服务类名 | Binder 名 | 一句话职责 |
|---|---|---|---|
| 1 | Watchdog | — | 死锁看门狗 |
| 2 | PlatformCompat | — | 平台兼容性框架 |
| 3 | FileIntegrityService | — | fs-verity 文件完整性 |
| 4 | **Installer** | installer | installd 客户端（dexopt） |
| 5 | DeviceIdentifiersPolicyService | — | 设备标识合规 |
| 6 | UriGrantsManagerService | uri | URI 授权 |
| 7 | PowerStatsService | — | 能耗统计 |
| 8 | **ATM + AMS** | activity | 调度中枢（书4） |
| 9 | DataLoaderManagerService | — | 增量安装 |
| 10 | IncrementalService | — | 增量数据加载 |

| 序 | 服务类名 | Binder 名 | 一句话职责 |
|---|---|---|---|
| 11 | **PowerManagerService** | power | 电源策略（书9） |
| 12 | ThermalManagerService | — | 热管理（书9） |
| 13 | HintManagerService | — | 性能暗示 |
| — | boot phase 100 | — | 等默认屏 |
| 14 | **PackageManagerService** | package | 包管理（书6） |
| 15 | UserManagerService | user | 多用户 |
| 16 | OtaDexoptService | — | OTA 后 dex 整理 |

### Core 批（轻量统计，:1336）

| 服务 | Binder 名 | 职责 |
|---|---|---|
| SystemConfigService | — | 系统配置 |
| BatteryService | batterymanager | 电池状态 |
| UsageStatsService | usagestats | 使用统计 |
| WebViewUpdateService | webviewupdate | WebView 版本 |
| CachedDeviceStateService | — | 设备状态缓存 |
| BinderCallsStatsService | — | Binder 调用统计 |
| RollbackManagerService | — | 升级回滚 |

### Other 批（大合唱 224 段，:1404-3059）

| 关键服务 | Binder 名 | 职责 | 所在册 |
|---|---|---|---|
| **InputManagerService** | input | 输入分发（书5） | |
| **WindowManagerService** | window | 窗口管理（书5） | |
| NetworkManagementService | network_management | 网络管理（书10） | |
| ConnectivityService | connectivity | 连接管理（书10） | |
| NotificationManagerService | notification | 通知（书13） | |
| LocationManagerService | location | 定位 | |
| AudioService | audio | 音频策略（书12） | |
| MediaSessionService | media_session | 媒体会话 | |
| BluetoothService | bluetooth_manager | 蓝牙 | |
| WifiService | wifi | Wi-Fi（书10） | |

| 关键服务 | Binder 名 | 职责 | 所在册 |
| --- | --- | --- | --- |
| TelephonyRegistry | telephony.registry | 电话状态（书19） | |
| MountService（vold 客户端） | mount | 存储（书11） | |
| JobSchedulerService | jobscheduler | 后台调度（书9） | |
| AlarmManagerService | alarm | 闹钟（书9） | |
| DeviceIdleController | deviceidle | Doze（书9） | |
| DropBoxManagerService | dropbox | 系统日志收集 | |
| …（200+ 服务） | … | … | … |

**设计洞察 · 逐段精读：为什么这张表重要**

面试时"说出 system_server 里 10 个服务及职责"是高频题——这张表就是答案。更重要的是**理解启动顺序=依赖顺序**：Installer 先于 AMS（AMS 装系统包要 dexopt）、PMS 先于 WMS（窗口要查包）。顺序错一个=启动失败。

# 第六章 层间通信机制总览

**设计洞察 · 学习目标**

1. 对比四种机制；2. 追踪属性set全链；3. 解释不对称设计

系统的“血管系统”由四条通道构成，各自解决不同问题：

| 机制 | 典型用途 | 数据形态 | 成本量级 |
| --- | --- | --- | --- |
| Binder IPC | 一切跨进程方法调用 | 结构化事务（Parcel） | 微秒级，一次拷贝 |
| System Properties | 全局状态广播（开机完成、USB 配置） | 键值字符串（mmap 共享读） | 纳秒级读 |
| Intent/ContentProvider | 应用间协作与数据共享 | 语义化消息/表格数据 | Binder 之上封装 |
| JNI | Java ↔ Native 边界 | 函数调用 | 纳秒级（经 ART 精心优化） |

属性系统值得单说：init 在第二阶段调用 `PropertyInit()`（`init.cpp:887`）建立 `/dev/__properties__` 共享区，所有进程从此 **mmap 只读**读取；写入则经 socket 连到 init 的属性服务（`property_service.cpp:527` 的 `handle_property_set_fd`），由 init 统一裁决（含 SELinux 检查）后广播。**读路径零系统调用、写路径集中仲裁**——这个不对称设计以极低成本支撑了全机数千项配置。

## 6.1 一次 SystemProperties.set 的完整下行链

把上一节的机制落到代码，从应用层下行到 init 的每一跳：

text 属性写入全链（自上而下）

```
应用：SystemProperties.set("debug.hwui.renderer", "skiagl")
↓ (frameworks/base/core/java/android/os/SystemProperties.java, native_set)
JNI: android_os_SystemProperties_set (core/jni/android_os_SystemProperties.cpp)
↓ __system_property_set
bionic: 发送 socket 到 /dev/socket/property_service (libc/bionic/system_properties.cpp)
↓ init 侧 accept4
init: handle_property_set_fd (property_service.cpp:527)
↓ 凭证 SO_PEERCRED + SELinux 上下文
仲裁：HandlePropertySet(:481) → PropertySet(:176) → 更新共享区 + 持久化(persist.)
↓ 属性变更通知
消费方：poll 属性 fd 或由 init 触发 on property:xxx=* 动作
```

对照第三册即将展开的 Binder 链，你会发现惊人相似的结构：**Java 门面 → JNI → Native 库 → 内核/守护进程仲裁 → 广播反馈**。Android 的“跨边界调用”几乎都是这个形状，差别只在仲裁者是谁（init 属性服务、Binder 驱动或 servicemanager）。

## 6.2 运行时资源覆盖（RRO）：不改代码的定制面

分区里的 overlay APK（书6 §9 扫描的 :320）在运行时由 **OverlayManagerService**（`services/core/.../om/OverlayManagerService.java`）挂载：把覆盖包的资源表叠加进目标包的 AssetManager——**按资源 ID 重定向**，同一 ID 优先取覆盖值。三层用途递进：厂商暗色主题/默认值定制（vendor overlay）、运行时主题切换（framework 的 android 覆盖）、用户个性化（ThemePicker）。它和 APEX 是一对：APEX 换"代码与库"，RRO 换"资源与配置"——都是"不动 system 分区本体"的定制哲学。读码入口：OMS 的 `setOverlayEnabled` 与 `OverlayActorEnumerator` 策略校验（谁能覆盖谁，受 targetName/签名约束）。

> **设计洞察 · 为什么属性不用 Binder**
> 属性读取发生在进程启动最早期（连 ART 都未就绪），也发生在 SELinux 策略加载前，必须是一个“dumb 到不能再 dumb”的机制。mmap 只读 + init 单点写，正是不依赖任何运行时的答案。

# 第七章 构建与产物体系

> **设计洞察 · 学习目标**
> 1. 画出boot.img组成；2. 理解Soong/Kati角色；3. 列出五大分区

## 7.1 分区布局

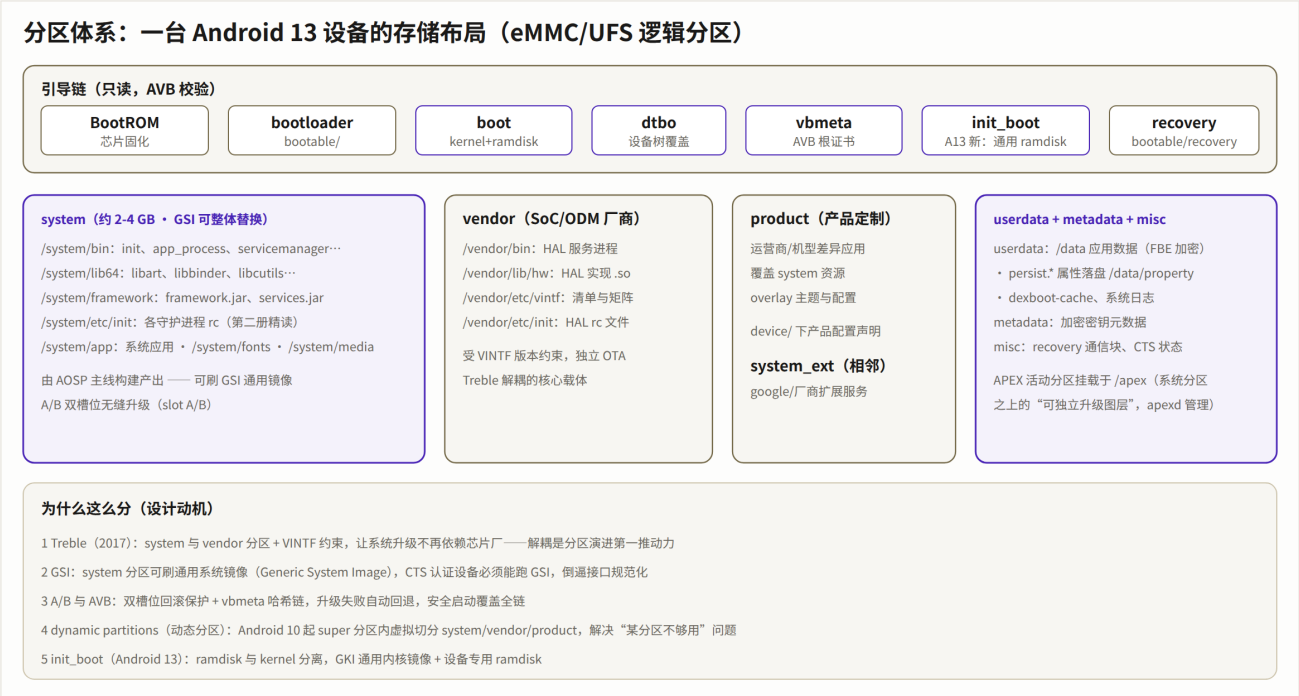



图 4 Android 设备分区布局：引导链、四大主分区与 data 系分区，附五大设计动机

分区的关键词是**解耦与可替换**：system 可刷 GSI 通用镜像，vendor 受 VINTF 接口矩阵约束独立升级，product 承载机型差异。Android 13 还引入 init_boot 分区，把通用 ramdisk 与内核分离，配合 GKI（通用内核镜像）。

## 7.2 构建流水线

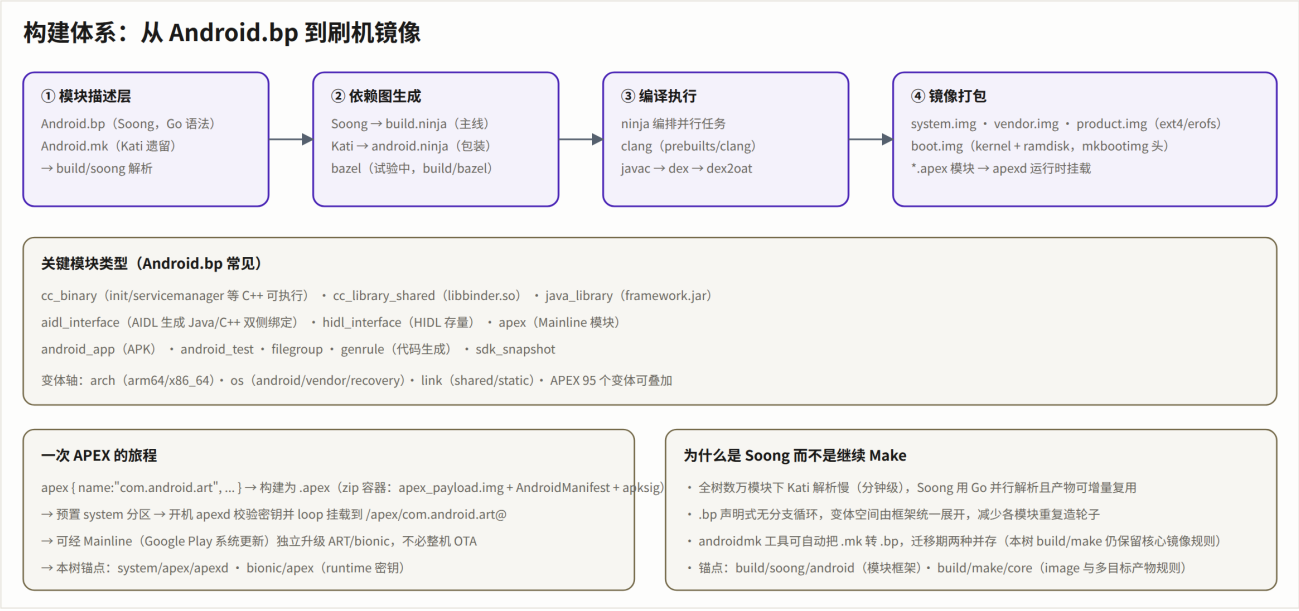



图 5 Soong/Kati→ninja→镜像的构建流水线与 APEX 生命周期、Soong 取代 Make 的动机

系统开发者的日常之一是读 `Android.bp`：`cc_binary` 产出守护进程，`aidl_interface` 从 .aidl 生成 Java/C++ 双侧绑定代码，apex 定义 Mainline 模块。**看一个模块的 Android.bp，就知道它编译成什么、装进哪个分区、暴露哪些接口**——这是改代码前的必做功课。

> **源码锚点 · 构建体系锚点**
> <font face="MonoFT" size="9">build/soong/android</font>：Soong 模块框架（Go）<br/><font face="MonoFT" size="9">build/make/core/Makefile</font>：system.img 等镜像组装规则（Kati 侧）<br/><font face="MonoFT" size="9">system/apex/apexd</font>：APEX 运行时守护（挂载 /apex）<br/><font face="MonoFT" size="9">bionic/apex/com.android.runtime.*</font>：runtime APEX 的 AVB 公钥与证书

## 7.4 从零构建一个系统 App（完整教程）

这是本册唯一一个端到端教程——从写代码到刷进设备。

### 第 1 步：写 App

text device/google/myproduct/apps/MyService/

```
├── Android.bp
├── AndroidManifest.xml
└── src/com/example/myservice/MainService.java
```

### 第 2 步：Android.bp

text Android.bp（完整模板）

```
android_app {
name: "MyService",
srcs: ["src/**/*.java"],
platform_apt: true, // 使用 platform 签名
privileged: true, // 安装到 priv-app
certificate: "platform", // platform 密钥
init_rc: ["MyService.rc"], // 随 App 安装 init 脚本
}
```

### 第 3 步：AndroidManifest.xml

xml AndroidManifest.xml（核心声明）

```
<manifest package="com.example.myservice"
android:sharedUserId="android.uid.system">
<application android:persistent="true"> <!-- 常驻 -->
<service android:name=".MainService"
android:exported="true" />
</application>
</manifest>
```

### 第 4 步：MyService.rc

text MyService.rc

```
service myservice /system/bin/app_process /system/bin com.example.myservice.MainService
class main
user system
group system
// 实际上 persistent App 由 AMS 启动，不走 init——
// 这里展示的是"非 persistent 版本"的 init 启动方式
```

### 第 5 步：声明到产品配置

text device/google/myproduct/device.mk

```
PRODUCT_PACKAGES += MyService
// 讲解注释：这一行告诉 Soong "构建时把 MyService 装进 system.img"
```

### 第 6 步：构建与部署

bash

```
source build/envsetup.sh
lunch aosp_x86_64-userdebug # 或你的产品 target
m MyService # 单模块构建（约 30 秒）
adb root && adb sync # 推送到设备
adb shell am startservice -n com.example.myservice/.MainService
# 或者看开机自启：
# adb shell getprop init.svc.myservice
```

### 第 7 步：验证

bash

```
adb shell dumpsys activity services com.example.myservice
# 应显示你的 Service 在运行
adb shell ps -A | grep myservice
# 应显示进程存在
```

> **设计洞察 · 逐段精读：这个教程串联了什么**
>
> 第2步的 Android.bp 是书15 构建系统的入口；第3步的 sharedUserId=system 涉及书6 的权限；第4步的 .rc 是书2 init 语言的实践；第5步的 device.mk 是产品配置；第6步的构建命令是 Soong 增量。**一个简单 App 的部署，串联了丛书五册的核心知识**——这就是"系统开发者"与"应用开发者"的区别：你不仅知道怎么写 App，还知道它怎么被构建、部署、启动。

# 第八章 运行时体系：ART 与 Zygote

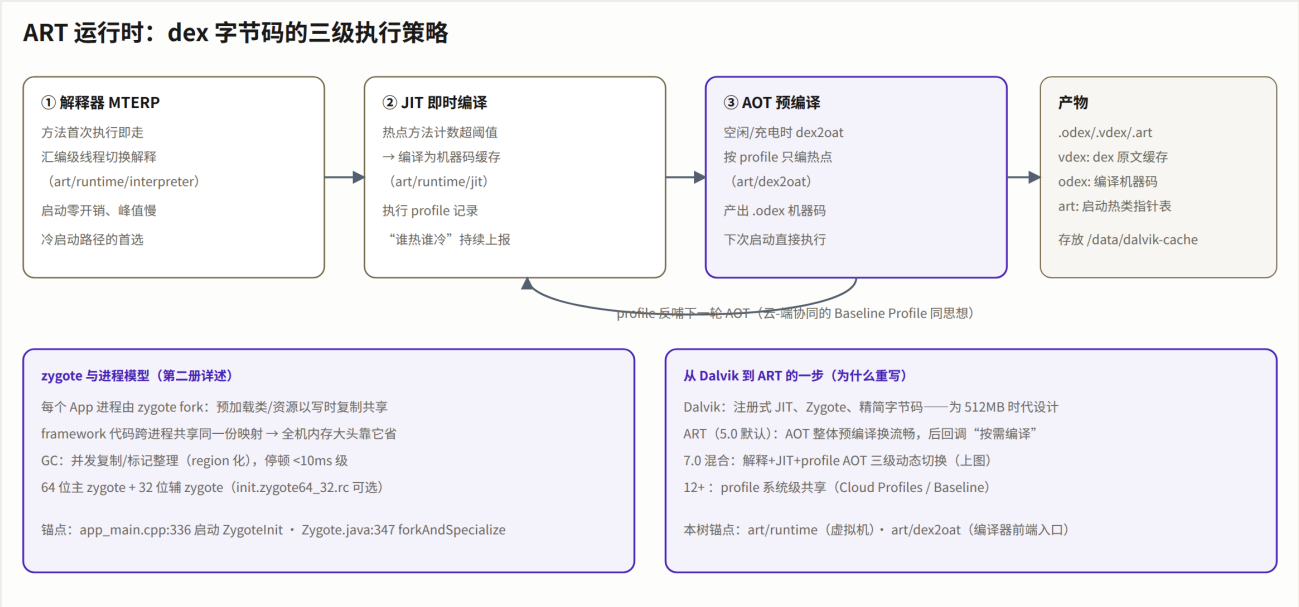



图 6 ART 三级执行策略（解释器/JIT/AOT + profile 反哺）与 zygote 进程模型、Dalvik→ART 演进动机

应用开发者视角的“Java”，在系统层是三件事：**dex 字节码规范、ART 虚拟机、Zygote 进程模型**。ART 的三级执行（解释器 → JIT → profile 引导的 AOT）让“安装快、启动快、运行快”这三个矛盾目标动态平衡。而 Zygote 把 `framework.jar` 与资源预加载进一个模板进程，之后所有 App fork 而出、共享只读页——第二册将从 `app_main.cpp` 开始走完这条链。

## 8.1 dex2oat 与启动镜像 boot.art

AOT 的输入输出由 `art/dex2oat` 定义：读入 .dex（vdex 中缓存原文）与 profile，产出 .odex（方法机器码）与 boot 镜像。**boot classpath**（framework.jar 等系统 jar 集）会额外编译为 `boot.art/boot.oat`——它们正是 Zygote 预加载时 mmap 的“共享机器码层”：所有 App 进程共享同一份框架类机器码映射，写时复制只在应用真正修改对象时发生。安装期与空闲期的编译策略由 `art/dex2oat/dex2oat.cc` 与 `system/serverconfiguration` 等协同控制；Cloud Profiles（Android 12+）让云端聚合的热点信息反哺本地 AOT。

## 8.2 ART GC 深潜：并发复制收集器

ART 的主收集器是**并发复制**（Concurrent Copying，CC）——art/runtime/gc/collector/concurrent_copying.cpp。这是 Android 性能的隐形基石。

### 为什么要并发?

text 停顿时间对比

```
Dalvik（标记-清除 STW）：200-500ms 停顿 → 13 年前的卡顿
ART 早期（标记-清除+部分并发）：50-100ms → 仍有感知
ART CC（读屏障并发搬移）：<5ms → 用户无感
// "并发"的意思：GC 与应用线程同时跑，只在小段时间暂停
```

### Region Space：堆的分区管理

text RegionSpace（art/runtime/gc/space/region_space.cc）

```
┌────┬────┬────┬────┬────┬────┬────┬────┐
| R0 | R1 | R2 | R3 | R4 | R5 | R6 | R7 | ← 256KB 固定大小 Region
| 已用|空闲|已用|已用|大对|空闲|已用|空闲|
└────┴────┴────┴────┴────┴────┴────┴────┘
// 讲解注释①：Region 是分配和回收的基本单位
// 新对象在当前分配 Region 顶部指针碰撞分配（纳秒级）
// 回收时整个 Region 的存活对象搬走 → Region 直接回收
// 讲解注释②：大对象（>3 个 Region 大小）走独立的 LargeObjectSpace
```

## 读屏障：并发搬移的核心

text 读屏障（Read Barrier）的原理

```
每个对象的引用读取都被插桩：
if (obj.hasForwardingPointer()) {
obj = obj.getForwardedObject(); // 搬到新位置
self.reference = obj; // 修正引用
}
// 讲解注释③：这就是"用 CPU 换延迟"——
// 每条引用读取多几条指令（约 3-5% 性能开销）
// 但 GC 搬移对象不需要暂停全部线程
// 每个线程在自己的读屏障里"顺路"搬移
```

## 三种屏障与协作

| 屏障 | 触发时机 | 做什么 |
| --- | --- | --- |
| 读屏障 | 每次读取引用 | 检查 forwarding 指针，按需搬移 |
| 写屏障 | 每次写入引用 | 记录跨 Region 引用（记忆集） |
| 标记屏障 | 标记阶段 | 灰→黑转换时的标记传播 |

**设计洞察 · 逐段精读：为什么面试常考 GC**

GC 是"用户感知不到但影响每一帧"的机制。掉帧排查时 FrameTimeline 显示 "GC pause" 是常见原因。理解 CC 的工作原理后，你能从 dumpsys meminfo 的各 space 数据推断出 GC 行为（Region 剩余少=即将 GC、LargeObjectSpace 大=有内存泄漏的大数组）。**GC 不是黑盒，它是可推理的机器。**

**源码锚点 · ART 关键产物与位置**

<font face="MonoFT" size="9">art/dex2oat</font>：AOT 编译器（读 dex+profile，产 odex）<br/><font face="MonoFT" size="9">/data/dalvik-cache</font>：运行期编译产物目录<br/><font face="MonoFT" size="9">boot.art / boot.oat</font>：框架类共享机器码（zygote 映射）<br/><font face="MonoFT" size="9">ZygoteInit.java:247 preloadClasses</font>：预加载类清单的消费者（第二册）

**设计洞察 · 为什么 App 进程必须从 Zygote fork**

若每个进程自行加载 200MB 级的类与资源，启动要数秒且内存爆炸。fork + 写时复制让“加载”只发生一次；代价是所有 App 共享同一个 ART 运行时版本——这也是 Mainline 化 runtime APEX 的原因之一：运行时可独立升级，但升级必须整机重启生效。

# 第十章 深入属性系统：一次 setprop 的完整旅程

属性系统是“最小的系统机制”，却五脏俱全——值得作为第一个逐函数精读的对象。

## 10.1 属性的三个存储域

- **默认域**：开机时 init 从 `/system/etc/prop.default`、`/vendor/build.prop`、`/data/property/persistent_properties` 等加载，见 `property_service.cpp:639` 的 `LoadProperties`。
- **共享内存区**：init 创建 `/dev/__properties__`，各域按前缀分区；所有进程 mmap **只读**。
- **persist 域**：`persist.` 前缀的属性由 `persistent_properties.cpp` 序列化到 `/data/property/persistent_properties`，重启不丢。

## 10.2 写入路径精读

任何进程调用 `SystemProperties.set`（bionic 的 `__system_property_set`）后发生的事：

cpp system/core/init/property_service.cpp:176（节选）

```
static uint32_t PropertySet(const std::string& name, const std::string& value,
std::string* error) {
size_t valuelen = value.size();
if (!IsLegalPropertyName(name)) { ... } // 1. 合法性：长度/字符集
if (valuelen >= PROP_VALUE_MAX) { ... }
if (StartsWith(name, "ctl.")) { // 2. ctl.* 是控制命令不是属性
return HandleControlMessage(name.c_str() + 4, value, getuid());
}
...
if (persistent_properties_loaded && StartsWith(name, "persist.")) {
error->insert(0, "Persistent property not loaded yet"); // 3. persist 未就绪前拒绝写
}
...
}
```

**设计洞察 · 逐段精读：这段 30 行代码里的四道闸**

①合法性（名字字符集/长度、值长度）——防注入与滥用；②`ctl.` 前缀劫持——属性写接口兼任服务控制通道（setprop ctl.start zygote 即可拉服务），但随后立即校验调用者权限；③persist 就绪检查——/data 未挂好前拒绝持久化，防止"丢在内存里装作持久化"；④`ro.` 只读——一次写后封死。四道闸合起来就是"一个最小 API 的完整安全叙事"。

三个细节值得一读：ctl. 前缀被劫持为**控制通道**（ctl.start/ctl.stop 直接驱动服务启停，等价于 setprop ctl.start zygote）；写入者身份（getuid）被记录；ro. 前缀只读属性在设置一次后返回 PREADONLY。

仲裁入口 HandlePropertySet（:481）先做 **SELinux source_context 检查**——不是任何进程都能写任何属性；通过 socket 收包的部分在 handle_property_set_fd（:527），它 accept 连接、读属性名与值、取对端凭证（SO_PEERCRED），再进入仲裁。**凭证来自内核，不可伪造**——这个模式 Binder 里将再次出现（第三册 tr.sender_euid）。

> **设计洞察 · 面试题常客：属性能跨 SELinux 域随便写吗**
> 不能。属性按前缀划分 SELinux 上下文（property_contexts 文件），写 sys.powerctl 需要对应域授权。这也是为什么“重启设备”属性普通 App 写不了——策略层早已拒绝。

## 10.3 读取路径：为什么读不要系统调用

bionic 的 __system_property_find 直接在 mmap 的属性区里按前缀定位域、二分查找。属性区布局与查找算法在 bionic/libc/bionic/system_property_api.cpp 与 system/core/property_service/libpropertyinfoparser 中——读路径没有任何内核参与，这就是“纳秒级读”的来源。应用侧的 SystemProperties.java 只是 JNI 薄封装。

## 6.3 属性系统完整走读（从 set 到生效的每一跳）

这是全书最详细的一段单机制走读——它展示了"一个看似简单的 API 背后有多少协作"。

### 第零跳：App 调用

java App 进程

```
SystemProperties.set("debug.hwui.renderer", "skiagl");
// 或 bionic 层：__system_property_set(name, value)
```

### 第一跳：bionic 的 stub

cpp bionic/libc/bionic/system_property_api.cpp

```
// 讲解注释①：bionic 是 C 库（App 的 native 层），不走 Binder——
// 因为属性操作可能在 Binder 就绪前发生（init 第二阶段）。
// 改为写 socket：/dev/socket/property_service
int __system_property_set(const char* name, const char* value) {
// 组包 → sendto(prop_sock) → 等 init 回复
}
```

### 第二跳：init 的属性服务

cpp system/core/init/property_service.cpp:527

```
static void handle_property_set_fd() {
int s = accept4(property_set_fd, ...);
// 讲解注释②：SO_PEERCRED 取调用者凭证（pid/uid）——内核背书
// 讲解注释③：SELinux source_context 检查
```

```
// → HandlePropertySet(:481) → PropertySet(:176)
}
```

## 第三跳：仲裁与写入

cpp PropertySet 的四道闸（:176 起）

```
static uint32_t PropertySet(const std::string& name, ...) {
// 闸1：合法性（名字字符集/长度）
// 闸2：ctl.* 前缀劫持为控制命令
// 闸3：persist.* 前缀 → 持久化到 /data/property/
// 闸4：ro.* 只读属性 → 已设则拒绝
// 通过全部 → 写入共享内存属性区 → 通知等待者
}
```

## 第四跳：共享区与读取

text 所有进程的"读"路径

```
init 创建 /dev/__properties__（mmap 共享区）
→ 所有进程 mmap 只读 → 零系统调用读取
→ bionic __system_property_find 按 prefix 分区二分查找
// 讲解注释④：这就是"纳秒级读"的来源——
// 读操作完全在用户态内存完成，不进内核
```

## 属性触发（反向链）

text 属性变化如何触发系统动作

```
init.rc 里的 on property:sys.boot_completed=1
→ init 的 ActionManager 匹配触发器
→ 执行 action 里的命令（如 start 某服务）
// 这就是"属性即状态机"——
// ctl.start zygote 等价于 setprop ctl.start zygote
```

**设计洞察 · 逐段精读：为什么这个走读重要**

属性系统是**最简单的跨进程机制**（比 Binder 简单得多），但它展示了系统设计的所有核心模式：**凭证由内核采集**（SO_PEERCRED）、**策略仲裁集中**（init 单线程）、**读写不对称**（mmap 读+socket 写）、**声明式触发**（on property）。理解了这四个模式，后面学 Binder、学 SELinux 都会轻松——它们是同一个设计哲学的不同表达。

**源码锚点 · 属性系统锚点速查**

<font face="MonoFT" size="9">init.cpp:887</font> PropertyInit：第二阶段建立属性区<br/><font face="MonoFT" size="9">init.cpp:912</font> StartPropertyService：打开 /dev/socket/property_service<br/><font face="MonoFT" size="9">property_service.cpp:176</font> PropertySet：写仲裁（合法性/ctl./persist）<br/><font face="MonoFT" size="9">property_service.cpp:527</font> handle_property_set_fd：socket 收包与凭证<br/><font face="CN-Med">SystemServer 启动后</font> sys.boot_completed=1（AMS.java:5149，第二册详述）

# 第十一章 SELinux：系统的第二套权限体系

传统 Linux 权限（DAC）回答“你是谁、文件给谁读写”；SELinux（MAC）回答“**你的域允许对什么类型做什么**”。Android 从 5.0 全量强制（enforcing），策略源码就在 `system/sepolicy/`。

## 11.1 从 init 的第一个动作看起

第二阶段 init 在属性初始化后立即做三件事（`init.cpp:901-903`）：`SelinuxSetupKernelLogging`、`SelabelInitialize`（加载 file_contexts）、`SelinuxRestoreContext`（`selinux.cpp:762`，按标签规则递归恢复全文件系统标签）。**在第一个服务启动前，规则就已就位**——此后每个服务进程 exec 时（`service.cpp` 的 `SetExecContext`）都会切入自己的 SELinux 域。

## 11.2 策略文件的分工

| 文件/目录 | 作用 |
| --- | --- |
| `system/sepolicy/public` | 导出给 vendor 使用的类型与接口（system_ext 同理） |
| `system/sepolicy/private` | system 私有策略 |
| `system/sepolicy/vendor` | vendor 侧策略默认集 |
| `*.te` 文件 | 域规则：allow/init neverallow，如 `allow init zygote_exec:file { read execute }` |
| `file_contexts` | 路径 → 文件类型标签（/system/bin/app_process64 → zygote_exec） |
| `service_contexts` | Binder 服务名 → 服务标签（activity → activity_service） |
| `property_contexts` | 属性前缀 → 属性标签 |

读懂三张 contexts 表，是排查“服务起不来/属性写不进/设备节点打不开”类问题的钥匙。而 `neverallow` 规则是策略的“宪法条款”——编译期强制，谁也不能悄悄放水。

一个最小的服务域定义长什么样（示意，风格取自 system/sepolicy）：

text 服务域示意（.te 片段）

```
type mysvc, domain; # 声明域类型
type mysvc_exec, exec_type, file_type; # 可执行文件类型
init_daemon_domain(mysvc) # 由 init 拉起并切入域
allow mysvc activity_service:service_manager find; # 允许查询 activity 服务
neverallow mysvc shell_data_file:file { read write }; # 永久禁止碰 shell 数据
```

读 avc denied 日志的方法：`avc: denied { find } for comm="mysvc" scontext=u:r:mysvc:s0 tcontext=u:object_r:activity_service:s0`——按 scontext/tcontext 反查

缺少的 allow 行，把最小权限补进策略。

> **设计洞察 · 系统开发者与 SELinux 的日常**
> 新增系统服务必须：定义 .te 域 + service_contexts 注册 + rc 文件声明 selinux 域。忘了会看到 avc denied 日志（logcat -b events | grep avc）。调试期可 setenforce 0 临时全局放开（仅 userdebug），但**任何 denied 都应通过策略显式 allow 解决**，而不是永久关闭。

# 第十二章 图形与输入骨架概览

渲染与输入是“系统开发者进阶深水区”，本册只画骨架坐标，细节留给后续批次。

**图形一侧的三个角色**：App 进程（`frameworks/base` 的 View 系统 + `libhwui`）把界面渲染进 BufferQueue 的图形缓冲；`system_server` 的 WMS 管理窗口层级与生命周期；`surfaceflinger`（`frameworks/native/services/surfaceflinger`）按 VSync 节拍把各窗口缓冲合成送显。三者之间传递的“货币”是 BufferQueue 里的 GraphicBuffer（DMA-BUF），控制流走 Binder，数据流走共享内存——**控制与数据分离**是图形栈的核心设计。

**输入一侧**：`InputReader/InputDispatcher`（`frameworks/native/services/inputflinger`）从内核 evdev 读取事件，经 `system_server` 的 `InputManagerService` 与 WMS 的窗口焦点信息，把事件注入焦点窗口的 socket。App 侧 `InputEventReceiver` 收到后再分发给 View 树。

**VSync 是整个图形栈的心跳**。SurfaceFlinger 从 HWComposer 获得硬件 VSync，软件分发（DispSync/VsyncSchedule）给两路客户：App 侧的 Choreographer（`frameworks/base/core/java/android/view/Choreographer.java`，驱动 doFrame 绘制节奏）与 SurfaceFlinger 自身的合成节拍。“App 按节拍生产、SF 按节拍消费”的三缓冲流水线，是 60/120Hz 流畅的基础；渲染专册将沿 VSync 一路深挖。

> **源码锚点 · 图形/输入锚点**
> <font face="MonoFT" size="9">frameworks/native/services/surfaceflinger/SurfaceFlinger.cpp</font>：合成主循环<br/><font face="MonoFT" size="9">frameworks/native/services/inputflinger/reader/InputReader.cpp</font>：事件读取<br/><font face="MonoFT" size="9">SystemServer.java:1570</font>：InputManagerService 创建（startOtherServices 阶段）<br/><font face="MonoFT" size="9">SystemServer.java:1586</font>：WindowManagerService.main 启动点

# 第十三章 观测与调试：把系统打开给你看

系统开发者的另一半功夫是**观测**。所有工具的原理都在源码里：

## 13.1 四件套

| 工具 | 看什么 | 源码原理 |
| --- | --- | --- |
| `adb shell service list` | 已注册 Binder 服务 | 向 servicemanager（句柄 0）枚举 mNameToService |
| adb shell dumpsys <服务> | 服务内部状态 | 调用服务的 dump() 方法——每个服务类自己实现（如 ActivityManagerService.dump） |
| `logcat` | 全机日志 | logd 汇聚 /dev/log/*，logcat 为客户端 |
| `perfetto` / `systrace` | 时间线与卡顿 | atrace 埋点（ATRACE_CALL 宏遍布本丛书精读过的启动代码） |

## 13.2 启动观测的具体落点

本丛书第二册的每个环节都有配套观测：`dmesg` 看 init 两阶段日志（"init second stage started!"，`init.cpp:839`）；`logcat -b events` 的 `boot_progress_*` 事件（如 `boot_progress_preload_start`，`ZygoteInit.java:887`）；`dumpsys activity services`、`dumpsys window` 验证服务与窗口状态；`sys.boot_completed` 属性作为“开机完成”的唯一权威信号（`ActivityManagerService.java:5149` 设置）。**先有观测，再谈优化**——这是系统性能工作的第一原则。

## 13.3 编译与实验环境

本树为完整可构建树。典型工作流：`source build/envsetup.sh` → `lunch`（选择目标，如 aosp_x86_64-userdebug）→ `m`（Soong 构建镜像）。虚拟设备 Cuttlefish（`device/google/cuttlefish`）可免真机做系统实验。日常单模块迭代用 `m services`（只编 services.jar）等目标 + `adb root; adb sync`。**本丛书只读研读，不改源码树**；但读者可以在自己的分支上练习。

`dumpsys` 的输出就是服务的自我陈述，例如：

text adb shell dumpsys power（节选）

```
POWER STATE (dumpsys power)
mWakefulness=Awake # 唤醒状态机当前值
mUserActivityTimeoutOverrideFromWindowManager=-1
mWakeLockSummary=0x1a # 活跃 wakelock 汇总
mUserActivitySummary=0x1
mHoldDisplaySuspendBlocker=true
mDisplay=Ready # 显示就绪标志
```

每个字段的赋值点都能在 `PowerManagerService.java` 中 grep 到——**dumpsys 输出即源码索引**，这是系统调试的高效路径：先 dumpsys 定位异常字段，再回源码找字段的所有写点。

（第十三章末的自检问题若都能回答，可径直进入第二册。）

**设计洞察 · 自检问题（试试不回看正文回答）**

1) 五层各自只依赖哪一层？2) servicemanager 在哪个目录，为什么它不需要注册就能被找到？3) system.img 与 vendor.img 的接口契约由什么文件保证？4) 一个属性从 set 到生效经过哪些裁决？5) 为什么 App 进程必须是 fork 的产物？——答不上来的小节值得重读。

# 第十四章 进阶路线图：本丛书之后怎么走




图 7 从应用开发者到系统开发者的三阶段路线、配套技能栈与自测里程碑

路线图给了三阶段与里程碑，这里补三句话原则：**第一，主线优先于专题**——先走完启动与 Binder 两条主线，再挑专题深潜，否则专题知识挂不住。**第二，输出倒逼输入**——每读完一册，合上书画一遍时序图/调用链，画不出的地方就是没懂的地方。**第三，让源码树成为日常**——把 riagrep 检索、dumpsys 验证变成肌肉记忆，系统开发者的核心竞争力不是背下架构，而是“面对陌生问题时的定位速度”。

丛书后续批次（AMS 与四大组件、WMS 与渲染栈、PMS 与安装流程）将沿用本册的写法：真实源码锚点 + 执行链路 + 设计决策。

# 第十五章 设计决策分析（全书总结）

本章把散落各章的“为什么”收拢成十条可复用的架构判断：

- **分层与依赖方向**：每层只依赖下一层，跨层只能走公共通道（Binder/属性）。反过来“哪层都不属于”的代码，就是放错位置的代码。

- **Binder 单一 IPC 抽象**：统一的安全模型（内核验 UID/PID）胜过多个“更快”但各搞一套安全机制的通道。
- **HAL 三代演进**：隔离（legacy→HIDL）→ 简化（HIDL→AIDL）。隔离是原则，双栈是代价，归一是终局。
- **分区即解耦**：Treble 用物理分区强制接口稳定，让 system 升级与 vendor 解耦；GSI/GKI 是同一思想的镜像/内核版。
- **APEX 模块化**：把“必须随系统走但又想独立升级”的部分（ART、bionic）打包成可挂载的 mini 分区。
- **system_server 集中 vs 关键服务独立**：集中减少 IPC 与状态同步成本；SurfaceFlinger/audioserver 独立是为崩溃隔离与实时性——边界画在“故障域”上。
- **SystemServiceRegistry 门面**：进程内缓存 Binder 代理，屏蔽服务获取细节，App/系统应用同一入口。
- **属性系统的不对称设计**：读极简（mmap）、写集中（init 仲裁），适配最早期的运行环境。
- **Zygote 预加载 + COW**：以一次加载换千百次 fork 的速度与内存，是 Android 多进程模型成立的前提。
- **SELinux 强制访问控制**：DAC 会 root 绕过，MAC 不允许；策略与代码同树（system/sepolicy），改服务必查策略。

# 第十六章 深水区专题一：SELinux 策略实战与 APEX 内部

## 16.1 SELinux 策略编写实战（读改两件套）

策略源码在本树的体量会刷新你的认知：仅 `system/sepolicy/private/` 下就有 **202 个 .te 文件**。编写实践的三板斧：

**text 给新服务 mysvc 补策略的标准动作**

```
① 定义域：mysvc.te 写 type mysvc, domain; type mysvc_exec, exec_type, file_type;
+ init_daemon_domain(mysvc)
② 声明清单：file_contexts 加 /system/bin/mysvc u:object_r:mysvc_exec:s0
service_contexts 加 mysvc u:object_r:mysvc_service:s0
③ 最小放行：按 avc denied 逐条补 allow——
neverallow 是"宪法"（domain.te:119/:137 起大量出现）：
编译期硬校验，任何 allow 与其冲突直接拒绝构建
④ 应用侧打标：seapp_contexts（sepolicy/private/seapp_contexts）
按 package name/uid 给 App 域（untrusted_app 等）
```

逐段精读：`audit2allow < avc.log` 能自动生成候选 allow，但**永远不要无脑采纳**——它只会"让报错消失"，不知道语义该不该放行。生产实践是先看 neverallow 是否已经替你拒绝了这条路（例如禁止普通域写 shell_data_file），再决定是改架构还是补权限。

## 16.2 APEX 激活与回滚内部（apexd）

> **源码锚点 · apexd 关键文件**
> <font face="MonoFT" size="9">system/apex/apexd/apexd_main.cpp:99</font> main 入口（init 的 post-fs-data 期拉起）<br/><font face="MonoFT" size="9">apex_file_repository.cpp</font>：收集 /system/apex 与 /data/apex（会话安装位）<br/><font face="MonoFT" size="9">apex_database.cpp</font>：活跃 APEX 挂载表（loop 设备与 /apex 视图）<br/><font face="MonoFT" size="9">apexd_session.cpp</font>：staged 会话与激活/回滚状态机

激活流程：开机 apexd 扫描 → 校验 AVB 公钥与版本 → loop 挂载到 `/apex/<name>@<ver>` → 类路径（apex_classpath.cpp）注入 ART。**回滚**：staged 会话默认"试运行"（boot flag），连续两次启动失败自动退回旧版——与会话状态机（session state：staged→activated→reckoned）一体，这就是 Mainline"敢独立发版"的底气。

## 16.3 Mainline 模块边界速查（哪些已模块化）

| 模块 | 内容 | 升级通道 |
| --- | --- | --- |
| com.android.art | ART 运行时与 dexopt（书6 §3.3 的 artd） | Google Play 系统更新 |
| com.android.runtime | bionic 与运行时库 | 同上 |
| com.android.tethering | 网络共享与连接模块 | 同上 |
| com.android.permission | PermissionController（权限 UI/逻辑） | 同上 |
| com.android.scheduling | JobScheduler 域逻辑 | 同上 |
| com.android.wifi | Wi-Fi 栈（部分） | 同上 |
| com.android.media / mediaprovider | 媒体编解码配置/存储提供者 | 同上 |
| （对照）framework/system 分区本体 | AMS/WMS/PMS 等核心服务 | 仅整机 OTA |

判断口诀：**"策略与 UI"可进模块，"事实与账本"留在系统**（与书6 权限对话框同一条分界线）。

# 第十七章 深水区专题二：Soong 实战与链接器命名空间

## 17.1 写一个 Android.bp 的完整心智

text 给书4 的假设服务 mysvc 写构建描述（Soong 语法要点）

```
cc_binary { // 产物：/system/bin/mysvc
name: "mysvc",
srcs: ["mysvc.cpp"],
shared_libs: ["libbinder", "libutils", "libcutils"], // 依赖→dt_needed
```

```
init_rc: ["mysvc.rc"], // 随二进制安装 rc（书2 的解析入口！）
vintf_fragments: ["mysvc.xml"], // HAL 时必须：VINTF 清单
relative_install_path: "hw", // 装到 /system/bin/hw（HAL 惯例）
proprietary: true, // 归入 vendor 分区（对应 soong 配置）
stem: "vendor.mysvc-default", // HAL 服务命名惯例
}
```

变体轴的直觉：Soong 会按 arch（arm64）×os（android/vendor）×link（shared/static）自动展开出多个构建目标——**你声明的是"家族"，构建系统生的是"全家福"**。改 bp 后 `m module-info` 可查模块间依赖图。

## 17.2 链接器命名空间与 linkerconfig

Android 的动态链接不是"一锅端"：**linker namespace**（配置源 `system/linkerconfig/`）按进程把 so 划入不同命名空间——vendor 进程只能见 vendor 命名空间导出的符号、framework 的 libbinder 与 vndbinder 版本互不可见（Treble 在**链接期**的隔离，比运行期 SELinux 更早）。bionic 与 glibc 的差异清单：缺 NSS/PAM、自带属性/日志 API、ABI 随 NDK 冻结——读码时遇到 `__system_property_get` 这类"陌生 libc 函数"，答案都在 bionic。

## 17.3 sanitizer 家族（内存安全工具链）

ASAN（插桩+影子内存，构建开关 sanitize: {address: true}）、HWASAN（ARMv8 TBI 硬件标签，内存开销小得多）、**MTE**（内存标签硬件化，书2 forkSystemServer 里 `arm64.memtag` 属性即开它）、GWP-ASan（低概率采样，生产期兜底）。系统服务默认带 GWP-ASan 抽奖（SystemServer.java:763 的 GWP_ASAN_LEVEL_LOTTERY 同源思想）——**内存安全不是单一工具，是分层的"概率×覆盖"组合拳**。

# 第十八章 边境机制三讲：JNI、Context 家族与 native 智能指针

前三册反复跨过 Java/Native 边境却未教过"过境规则"。本章补上三门必修前置课。

## 18.1 JNI：两种注册与三类引用

**注册**：静态（Java 声明 native、库加载时 `RegisterNatives` 显式绑定——框架全部走此路，`AndroidRuntime.cpp` 的 startReg 一次注册几百个）与动态（`JNI_OnLoad` 里按签名查找，JNI 调试器友好但慢）。**引用三态**：局部引用（方法栈内，帧返回即回收——**循环里不释放会撑爆表**）、全局引用（跨线程/跨调用持有，`NewGlobalRef/DeleteGlobalRef` 成对）、弱全局（不阻止 GC，用前必须 `GetObjectLocalRef` 升格检查）。第三册 `javaObjectForIBinder` 里 BinderProxy 的 nativeData 长期持有就是全局引用的现场；`JavaBBinderHolder` 的弱缓存（:462-485）则是弱引用的现场。**异常语义**：JNI 异常不中断 native 执行，必须显式 `ExceptionCheck` 后返回——书3 的 transact JNI 里每一跳都有这个检查。

## 18.2 Context 家族：一把钥匙配千把锁

Context 是"访问系统环境的句柄"，族谱三支：**ContextImpl**（`core/java/android/app/ContextImpl.java:192`，唯一实现，持有 LoadedApk/权限主题/显示区）——**ContextWrapper**（纯委托壳，Service/Application 都是它的子类）——以及各组件 Context（Activity 的还带窗口与主题注入）。系统侧的"系统 Context"（书2 createSystemContext :882）是给 system_server 里那些"没有界面的应用"配的 ContextImpl。理解两件事就够用：**getSystemService 的缓存挂在 ContextImpl 上**（书1 §5.2 的 fetcher 缓存）；**startActivity 的"令牌"来自 Context 背后的窗口/Activity**——用 Application Context 启动有 UI 依赖的 Activity 会失败，就是令牌缺失的经典报错。

## 18.3 sp/wp/RefBase：native 世界的 GC

libutils 自带一套引用计数 GC（`system/core/libutils/include/utils/RefBase.h:281`）：**sp<T>** 强指针（构造 incStrong/析构 decStrong，计数归零销毁）、**wp<T>** 弱指针（只升弱计数，-promote() 升格检查对象是否存活——**native 世界的"软引用"**）。为什么不用裸指针：Binder 世界对象跨线程跨进程共享（书3 的 BBinder/BpBinder 全是 RefBase 子类），需要确定性生命周期；为什么不用 std::shared_ptr：历史先于标准库成熟，且弱引用语义可自定义。第三册的 BC_ACQUIRE/BC_INCREFS 命令，本质是**把这套计数跨越进程镜像**——两端各自 sp，驱动保证两本账同步。读懂 RefBase，书3 §5.4 的引用计数命令族就从"名词"变成"机制"。

> **源码锚点 · 本章锚点**
> <font face="MonoFT" size="9">frameworks/base/core/jni/AndroidRuntime.cpp</font> startReg（静态注册大表）<br/><font face="MonoFT" size="9">core/java/android/app/ContextImpl.java:192</font> / <font face="MonoFT" size="9">LoadedApk.java:107</font><br/><font face="MonoFT" size="9">system/core/libutils/include/utils/RefBase.h:281</font><br/>案例：<font face="MonoFT" size="9">android_util_Binder.cpp:749</font>（全局引用）/<font face="MonoFT" size="9">:462</font>（弱缓存）

# 附录 A 版本指纹与阅读路线

**版本指纹**（`build/make/core/version_defaults.mk`）：`DEFAULT_PLATFORM_VERSION := TP1A`；`PLATFORM_SDK_VERSION := 33`；`PLATFORM_SECURITY_PATCH := 2022-08-05`。即 Android 13 Tiramisu 初始版本。

**下一步**：第二册《系统启动主线》将从内核拉起 init 的第一行代码开始，沿 `init → Zygote → SystemServer → Launcher` 逐函数走完开机全程；第三册《Binder IPC 全链路剖析》将把本册反复出现的“经 Binder 调用”逐跳展开。三册读完，你面对任何 Android 系统问题，都能定位到“哪一层、哪个进程、哪条调用链”。

# 附录 B 系统进程归属速查

| 进程 | pid 典型 | 出生方式 | 一句话职责 |
| --- | --- | --- | --- |
| init | 1 | 内核启动 | 万物之父，rc 解释器与服务监护 |
| ueventd | 2 | init fork（argv[0] 切换） | uevent 处理、/dev 节点与固件加载 |
| servicemanager | - | init 服务 | Binder 服务名录 |
| zygote（主/辅） | - | init 服务 | App 与 system_server 之母 |
| system_server | - | zygote fork | 160+ 框架服务宿主 |
| surfaceflinger | - | init 服务 | 图形合成 |
| audioserver | - | init 服务 | 音频策略与混音 |
| cameraserver | - | init 服务 | 相机 HAL 宿主 |
| mediaserver | - | init 服务 | 媒体编解码 |
| netd | - | init 服务 | 网络配置执行者 |
| vold | - | init 服务（early-fs） | 存储卷与加密 |
| lmkd | - | init 服务 | 低内存查杀 |
| logd | - | init 服务 | 日志中枢 |
| bootanimation | - | init 服务（core animation） | 开机动画 |
| adbd | - | init 服务 | 调试桥（user 版受限） |
| installd | - | init 服务 | APK 安装/dexopt 执行者 |
| *.vendor HAL 进程 | - | init 服务（class hal） | 硬件服务（vendor 分区） |

# 附录 C 常用系统属性速查（带源码出处）

| 属性 | 含义 | 设置/读取位置 |
| --- | --- | --- |
| ro.build.version.sdk | SDK 版本（33） | 构建期写入，build.prop |
| ro.product.cpu.abilist | CPU ABI 列表 | 决定 zygote 单/双形态 |
| ro.bootmode | 启动模式（charger=充电） | init.cpp:978 分支触发链 |
| ro.crypto.state | 加密状态（encrypted/unencrypted） | zygote-start 触发条件 |
| sys.boot_completed | 开机完成权威标志 | AMS.java:5149 设置 |

| 属性 | 含义 | 设置/读取位置 |
| --- | --- | --- |
| init.svc.zygote | 服务状态机（running/restarting/stopped） | service.cpp:684 NotifyStateChange |
| persist.sys.timezone | 时区 | SystemServer.run:777 校正 |
| ctl.start / ctl.stop | 服务控制命令 | property_service.cpp:185 分流 |
| debug.sf.* | SurfaceFlinger 调参 | SF 读取 |
| dalvik.vm.* | ART 参数（heapsize 等） | libart 读取 |

# 附录 D 术语表

| 术语 | 展开 | 一句话解释 |
| --- | --- | --- |
| AOSP | Android Open Source Project | Android 开源项目及其源码树 |
| ART | Android Runtime | 5.0 起的运行时，替代 Dalvik |
| APEX | Android Pony EXpress | 可独立升级的模块容器（.apex） |
| AVB | Android Verified Boot | 镜像完整性校验链 |
| CTS | Compatibility Test Suite | 兼容性认证测试集 |
| GKI | Generic Kernel Image | 通用内核镜像（init_boot 分离配套） |
| GSI | Generic System Image | 可刷的通用 system 镜像 |
| HAL | Hardware Abstraction Layer | 硬件抽象层 |
| HIDL | HAL Interface Definition Language | Treble 时代的 HAL IDL（冻结维护） |
| JIT/AOT | Just/Ahead Of Time | 即时/提前编译 |
| LMKD | Low Memory Killer Daemon | 用户态低内存查杀 |
| MAC/DAC | Mandatory/Discretionary Access Control | 强制/自主访问控制 |
| PMS | PackageManagerService | 包管理服务 |
| PSI | Pressure Stall Information | 内核压力信息（lmkd 输入） |
| RC | Android Init Language | .rc 配置文件（init 语言） |
| SEPolicy | SELinux Policy | 强制访问控制策略 |
| Treble | Project Treble | system/vendor 架构解耦工程 |
| USAP | UnSpecialized App Process | Zygote 预孵化进程池 |
| VINTF | Vendor Interface | 设备接口清单与兼容矩阵 |
| WMS | WindowManagerService | 窗口管理服务 |

# 附录 E 上手实验（半天量）

实验一“服务普查”：`adb shell service list` 统计服务总数，找出十个你认识的与十个陌生的；陌生的逐一 `dumpsys` 看自述，猜职责再回源码验证（services/core 对应目录）。

实验二“属性观察”：`adb shell getprop | wc -l` 数属性量级；`getprop init.svc.zygote`、`getprop sys.boot_completed`、`getprop ro.build.version.sdk` 各自对应本册哪一章；用 `setprop` 试试写 `ro.` 前缀，观察拒绝行为（property_service.cpp:176 的只读检查）。

实验三“SELinux 初体验”（userdebug）：`adb shell getenforce` 看模式；`setenforce 0` 前后各跑一次某功能，`logcat -b events | grep avc` 收集差异；把一条 denied 翻译成策略语言（对照第十一章 11.2 的表）。

实验四“目录定向”：不看本册，凭记忆说出 frameworks/native/cmds/servicemanager、system/core/rootdir、frameworks/base/services/core/java/com/android/server/wm 各住着什么；再全书检索验证。

每个实验写三行结论：看到了什么、对应哪条源码锚点、哪里出乎意料。

# 教科书习题集（分级）

## 第一级：简答题（检验概念理解）

1. Android 软件栈分为哪五层？每层只依赖哪一层？

2. "贯穿列"的三条机制是什么？各自解决什么问题？

3. 为什么本源码树的 kernel/ 目录只有三个子目录？AOSP 的内核源码在哪？

4. HAL 从 legacy 演进到 HIDL 的动机是什么？HIDL 到 AIDL 又解决了什么？

5. system_server 中为什么不把 SurfaceFlinger 放进来？

6. Boot.img 和 init_boot 分区各自装什么？GKI 改变了什么？

7. ART 的三级执行策略是什么？谁决定一个方法用哪级？

8. APEX 模块化解决了什么问题？哪些模块适合 APEX 化？

9. 属性系统的"读快写慢"是什么意思？为什么这么设计？

10. Soong 为什么取代了 Make？两者的根本区别是什么？

## 第二级：分析题（检验机制理解）

11. **追踪调用链**：画出 context.getSystemService(Context.WIFI_SERVICE) 从 App 进程到 system_server 的完整路径，标出每次进程/线程切换。

12. **架构推演**：如果 Google 决定把 SystemUI 从独立进程移到 system_server 内，会带来什么问题？至少给出三条。

13. **对比分析**：Binder 与 Unix socket 都是 IPC，为什么 Android 选了 Binder？从安全、性能、能力传递三个维度对比。

14. **机制拆解**：preload() 加载了哪些内容到 Zygote？fork 后子进程如何继承？写时复制在这里起了什么作用？

15. **失败场景**：如果 boot.img 的 ramdisk 损坏，启动会在哪一步失败？系统会怎样？

16. **安全推演**：一个恶意 App 想伪造 getCallingUid() 返回 system uid（1000），在当前 Binder 架构下可能吗？为什么？

17. **版本演进**：Android 9 的 ClientTransaction 改造解决了什么问题？改造前后有什么区别？

## 第三级：设计题（检验系统思维）

18. **设计新机制**：假设你要设计一个"系统级配置变更通知"机制（比属性更实时、比 Binder 更轻量），你会怎么设计？考虑推/拉模型、安全、性能、向后兼容。

19. **架构决策**：你要给一个新 SoC 平台移植 Android，列出前十个必须实现的 HAL 及优先级。

20. **排障设计**：用户报告"开机后 30 秒才出现桌面"，给出分层排查方案（从内核到 Framework），每层至少一个检查命令。

## 参考答案要点

**1.** 应用层-Framework-Native-HAL-内核；每层只依赖正下方一层。

**5.** 崩溃隔离+渲染实时性+内存配额独立（SF 的 SCHED_FIFO 线程不该跟 AMS 抢核）。

**11.** App ContextImpl->SystemServiceRegistry Fetcher->ServiceManager.getService("wifi")->Binder十跳（书3）->system_server WifiService->BinderProxy->WifiManager->缓存。

**13.** 安全：Binder驱动采集UID/PID不可伪造 vs socket自报；性能：一次拷贝 vs 两次；能力：flat_binder_object传引用 vs 只传数据。

**15.** 第一阶段init无法挂载system分区->switch_root失败->内核panic或bootloop。

**19.** 前三决定能不能开机：Power>Health>Graphics(composer)>Audio>Camera>Sensors>Boot>Keymaster>Gatekeeper>DRM。

# 术语表（按拼音排序）

| 术语 | 全称 | 所在册 | 一句话解释 |
| --- | --- | --- | --- |
| APEX | Android Pony EXpress | 册1 | 可独立升级的模块容器（.apex），运行时挂载到 /apex |
| ART | Android Runtime | 册1 | Java 运行时：解释/JIT/AOT 三级执行 |
| AVB | Android Verified Boot | 册1/2 | 镜像完整性校验链（vbmetro->boot/system/vendor） |
| Binder | -- | 册3 | 核心 IPC：一次拷贝+内核身份+能力传递 |
| BootConfig | Boot Configuration | 册2 | kernel cmdline 的结构化继任者 |
| BSP | Board Support Package | 册14 | 板级支持包（内核+设备树+HAL+sepolicy+固件） |
| COW | Copy-On-Write | 册1/2 | fork 后共享物理页，写时才复制 |
| CTS | Compatibility Test Suite | 册18 | 兼容性测试，通过=Google Play 预装权 |
| DE/CE | Device/Credential Encrypted | 册11 | FBE 双域：开机即解 vs 解锁后解 |
| dm-verity | Device Mapper Verity | 册1/6 | 运行时只读分区块级校验 |
| FBE | File-Based Encryption | 册11 | 文件级加密（按目录分域） |
| GKI | Generic Kernel Image | 册2 | 通用内核镜像：boot 只装 Google 签名内核 |
| HAL | Hardware Abstraction Layer | 册1/14 | 硬件抽象层（legacy->HIDL->AIDL 三代） |
| HIDL | HAL Interface Definition Language | 册1/14 | Treble 时代的 HAL IDL（已冻结） |
| init.rc | Android Init Language | 册2 | init 配置文件语法（service/on/import） |
| JIT | Just-In-Time | 册1 | 运行时热点编译（ART 三级之一） |
| KeyMint | -- | 册11 | 硬件密钥管理 HAL（TEE/StrongBox） |
| Mainline | Android Mainline | 册1 | Google Play 系统更新机制（APEX 模块化） |
| MTE | Memory Tagging Extension | 册2/4 | ARMv8.5 硬件内存标签 |
| NMS | NotificationManagerService | 册13 | 通知管理（排序/通道/速率） |
| PMS | PackageManagerService | 册6 | 包管理（扫描/安装/权限） |

| 术语 | 全称 | 所在册 | 一句话解释 |
| --- | --- | --- | --- |
| RIL | Radio Interface Layer | 册19 | 电话框架与 modem 的抽象层（socket） |
| RRO | Runtime Resource Overlay | 册1 | 运行时资源覆盖 |
| SoC | System on Chip | 册1/14 | 片上系统（CPU+GPU+ISP+modem） |
| Soong | -- | 册15 | Android 构建系统（Android.bp->ninja） |
| TF | Tradefed | 册18 | 测试调度框架 |
| Treble | Project Treble | 册1 | system/vendor 架构解耦工程 |
| UAPI | User API | 册1/3 | 内核对用户空间的稳定接口 |
| VHAL | Vehicle HAL | 册16 | 车机 HAL（车速/档位等属性模型） |
| VINTF | Vendor Interface | 册1/14 | 设备接口清单与兼容矩阵 |
| VTS | Vendor Test Suite | 册18 | HAL 接口验证套件 |
| WMS | WindowManagerService | 册5 | 窗口管理（层级/焦点/Insets） |
| Zygote | -- | 册2 | 应用进程之母（预加载+COW fork） |

# 排障手册（按"我的问题是..."索引）

## 开机慢

- dmesg 检查 init 各阶段耗时
- logcat -b events 看 boot_progress 时戳
- dumpsys activity 查启动完成状态
- 深入：书2（启动主线）+ 书9（后台调度）

## 服务找不到

- service list 确认是否注册
- getprop init.svc.XXX 检查 init 状态
- dumpsys -l 查 dumpsys 可见服务
- 深入：书3（Binder）+ 书14（VINTF 检查）

## 没声音

- dumpsys media.audio_flinger 看 Track 状态
- dumpsys media.audio_policy 看路由

- 深入：书12（音频管线）

## 通知不显示

- dumpsys notification 查排名与原因
- 检查勿扰/通道/桶限制
- 深入：书13（通知管线）

## 界面卡顿

- dumpsys gfxinfo 看帧统计
- perfetto FrameTimeline 看责任段
- dumpsys SurfaceFlinger --latency 看 SF 时戳
- 深入：书5（Choreographer）+ 书8（渲染）

## 应用被杀

- logcat -s lmkd 看击杀记录
- dumpsys activity processes 看 adj/state
- 深入：书4（进程优先级）+ 书9（后台）

## 网络不通

- ping 基础连通
- dumpsys connectivity 看评分
- dumpsys dnsresolver 看 DNS
- 深入：书10（网络栈）

## 编译太慢

- m nothing 验证语法
- m 单模块增量
- adb sync 增量部署
- 深入：书15（构建系统）